

Formation en html5/css3

ORANGE DIGITAL CENTER

.....

Thierno Oumar Sylla Bah : Assistant Formation ODC



Jour 1: HTML - Structure d'une page web

- **Introduction:**

- Historique
- Le web et les langages front-end
- Rôle du **HTML** et du **CSS**

- **Structure d'une page HTML:**

- Balises **HTML** de base (`<!DOCTYPE>`, `<html>`, `<head>`, `<body>`)
- Contenu textuel (`<p>`, `<h1>`, `<h2>`, etc.)
- Liens hypertexte (`<a>`)
- Images (`<img>`)
- L'imbrication des balises et des éléments
- Les listes

- **Les formulaires**

Jour 2: CSS - Mise en forme des pages web

- **Introduction au CSS:**

- Rôle du CSS dans la mise en forme
- Sélecteurs CSS pour cibler les éléments HTML
- Propriétés CSS pour définir les styles

- **Mise en forme du texte:**

- Polices de caractères, tailles et couleurs
- Gras, italique et souligné
- Marges et paddings

- **Mise en forme des liens:**

- Couleurs, styles et décorations
- Effets de survol (hover)

- **Organisation des éléments:**

- Disposition en bloc et en ligne
- Flottants et positionnement

- **Exercices:**

- Mise en forme d'une page web avec CSS
- Application de styles aux titres, paragraphes et liens
- Création d'une mise en page attractive

Jour 3: La maitrise du css

1 - Les sélecteurs CSS complexes et combinaisons

- Sélectionner tous les éléments avec le sélecteur CSS universel ou sélecteur étoile (*)
- Appliquer des styles à plusieurs éléments avec le caractère virgule (,)
- Utiliser plusieurs sélecteurs CSS à la suite
- Appliquer des styles aux enfants directs d'un autre élément
- Sélectionner l'élément suivant directement un autre élément en CSS
- Sélectionner tous les éléments suivant un autre élément en CSS

2 - Ordre d'application (cascade) et des règles en CSS

- Le mécanisme de cascade CSS
- L'ordre de déclaration dans le code.

Jour 4: structure d'un site web

- Structuration d'un site web:

- Utilisation de balises **HTML** pour organiser le contenu (**header**, **nav**, **section**, **article**, **footer**)
- Création de menus de navigation

- Responsive design:

- Principes de base pour adapter le site aux différents écrans
- Media queries pour définir des styles spécifiques

- Projet à réaliser:

- Choix du projet

- Outils et ressources:

- Présentation d'outils de développement web (éditeurs de code, inspecteur web)
- Liens vers des tutoriels et ressources en ligne pour approfondir ses connaissances
<https://www.w3schools.com/>

Jour 1: HTML - Structure d'une page web

1. INTRODUCTION:

• Les différentes versions de HTML et CSS

Au fil du temps, les langages **HTML** et **CSS** ont beaucoup évolué. Dans la toute première version de **HTML** (**HTML 1.0**) il n'était même pas possible d'afficher des images !

Voici un très bref historique de ces langages pour votre culture générale.

1. Les versions de HTML

HTML1 : c'est la toute première version créée par **TimBerners-Lee** en 1991.

HTML2 : la deuxième version du **HTML** apparaît en 1994 et prend fin en 1996 avec l'apparition du **HTML 3.0**. C'est cette version qui posera en fait les bases des versions suivantes du HTML. Les règles et le fonctionnement de cette version sont donnés par le W3C (tandis que la première version a été créée par un seul homme).

HTML3 : apparue en 1996, cette nouvelle version du HTML rajoute de nombreuses possibilités au langage comme les tableaux, les applets, les scripts, le positionnement du texte autour des images, etc.

HTML4 : il s'agit de la version la plus répandue du **HTML** (plus précisément, il s'agit de **HTML 4.01**). Elle apparaît pour la première fois en 1998 et propose l'utilisation de frames (qui découpent une page web en plusieurs parties), des tableaux plus complexes, des améliorations sur les formulaires, etc. Mais surtout, cette version permet pour la première fois d'exploiter des feuilles de style, notre fameux **CSS** !

HTML5 : c'est la dernière version. Elle apporte de nombreuses améliorations comme la possibilité d'inclure facilement des vidéos, un meilleur agencement du contenu, de nouvelles fonctionnalités pour les formulaires, etc. C'est cette version que nous allons découvrir ensemble.



1. Les versions de CSS

CSS 1 : dès 1996, on dispose de la première version du **CSS**. Elle pose les bases de ce langage qui permet de présenter sa page web, comme les couleurs, les marges, les polices de caractères, etc.

CSS 2 : apparue en 1999 puis complétée par **CSS 2.1**, cette nouvelle version de **CSS** rajoute de nombreuses options. On peut désormais utiliser des techniques de positionnement très précises, qui nous permettent d'afficher des éléments où on le souhaite sur la page.

CSS 3 : c'est la dernière version, qui apporte des fonctionnalités particulièrement attendues comme les bordures arrondies, les dégradés, les ombres, etc.



Le web et ses langages

- Le web, ou World Wide Web, est un système d'information mondial accessible à tous via internet. Il est composé de pages web reliées entre elles par des hyperliens. Ces pages web permettent d'afficher du texte, des images, des vidéos et d'autres contenus multimédias.

Les langages front-end

- Pour créer et afficher des pages web, on utilise des langages de programmation spécifiques appelés langages front-end. Les langages front-end les plus populaires sont :
- **HTML (HyperText Markup Language)**: Ce langage est utilisé pour structurer le contenu d'une page web, en définissant les titres, paragraphes, images, liens, etc.
- **CSS (Cascading Style Sheets)**: Ce langage est utilisé pour définir l'apparence visuelle d'une page web, en gérant les couleurs, les polices, la mise en page et le responsive design.

- **JavaScript:** Ce langage permet d'ajouter des fonctionnalités interactives aux pages web, comme des animations, des jeux, des formulaires et des interactions avec l'utilisateur.

Comment fonctionnent les langages front-end

- Lorsque vous visitez une page web, votre navigateur web interprète le code **HTML**, **CSS** et JavaScript pour afficher la page et exécuter ses fonctionnalités.
- Le code **HTML** définit la structure de la page et le contenu à afficher.
- Le code **CSS** définit l'apparence visuelle de la page, comme les couleurs, les polices et la mise en page.
- Le code JavaScript ajoute des fonctionnalités interactives à la page, comme des animations, des jeux et des formulaires.

Apprendre les langages front-end

- De nombreuses ressources sont disponibles pour apprendre les langages front-end, comme des tutoriels en ligne, des livres et des formations. Il est important de commencer par les bases du **HTML** et du **CSS** avant de passer à **JavaScript**.

Avantages d'apprendre les langages front-end

- Les langages front-end sont des compétences très demandées sur le marché du travail. Ils vous permettent de créer des sites web, des applications web et des interfaces utilisateur interactives. Apprendre les langages front-end est un excellent moyen de développer vos compétences en programmation et d'ouvrir de nouvelles opportunités de carrière.

Structure d'une page HTML

- Une page **HTML** est composée de plusieurs éléments qui définissent la structure et le contenu de la page. La structure de base d'une page **HTML** est la suivante :



La structure de base en HTML

<> index.html X

<> index.html > ...

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>ODC-GUINEE</title>
7  </head>
8  <body>
9
10 </body>
11 </html>
12
```

Détail de la structure:

- **<!DOCTYPE html>** : Déclaration du type de document HTML5.
- **<html>** : Balise racine qui englobe l'ensemble du contenu de la page.
- **<head>** : En-tête de la page qui contient des informations métadonnées sur la page.
- **<title>** : Titre de la page affiché dans le titre du navigateur.
- **<meta charset = "utf-8 ">** : Déclaration du **charset** de la page (ici, **UTF-8** pour l'encodage des caractères).
- **<body>** : Corps de la page qui contient le contenu visible pour l'utilisateur, cela peut être du texte, des images, des vidéos, des jeux, des pistes audio jouables, et ainsi de suite.



La structure de base en HTML

<!DOCTYPE>

<html>

<head>

`<meta charset="utf-8">`

`<title> Mon titre </title>`

</head>

<body>

</body>

</html>

Création d'une première page HTML simple

Pour créer une page web en **HTML**, il suffit de créer un fichier ayant l'extension **.html**. Ce fichier s'ouvre dans le navigateur web simplement en faisant un double-clic dessus.

Le **HTML** est un langage de balisage, c'est-à-dire un langage qui va servir à définir chaque contenu dans une page.

Avant de créer une page web en **HTML**, on va d'abord aborder les notions de base à connaître comme les **éléments**, **balises**, **attributs**, **imbrication** et **indentation HTML**.

Les éléments HTML

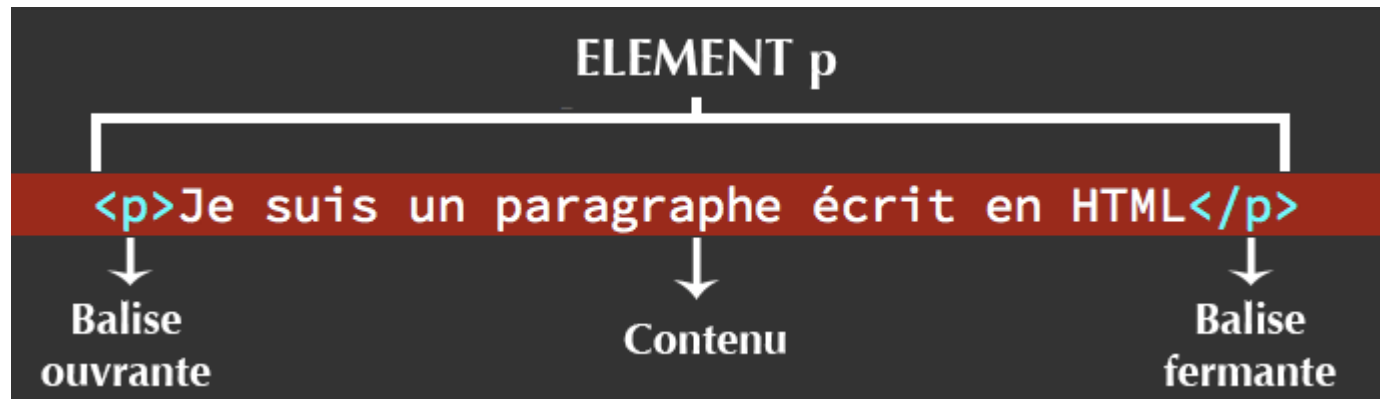
- Le langage **HTML** tout entier repose sur l'utilisation d'éléments. Si vous comprenez bien ce qu'est un élément, vous comprenez le **HTML**.
- Les éléments **HTML** vont nous permettre de créer la structure d'une page **HTML**, de définir chaque contenu de notre page et également de passer certaines informations utiles au navigateur pour afficher la page (comme le type d'encodage utilisé par exemple pour que celui-ci affiche bien nos accents français).

- Dans une page, nous allons utiliser les éléments en **HTML** pour marquer du contenu, c'est-à-dire pour lui donner du sens aux yeux des navigateurs et des moteurs de recherche. Selon l'élément utilisé, un navigateur va reconnaître le contenu comme étant de telle ou telle nature.
- Ainsi, on va utiliser des éléments pour définir un paragraphe ou un titre par exemple, ou encore pour insérer une image ou une vidéo dans un document.
- L'élément **p**, par exemple, sert à définir un paragraphe, tandis que l'élément **a** va nous permettre de créer un lien, etc.
- Aujourd'hui, il existe plus de 120 éléments **HTML** différents aux rôles très variés et qui font la richesse de ce langage.

Les balises HTML

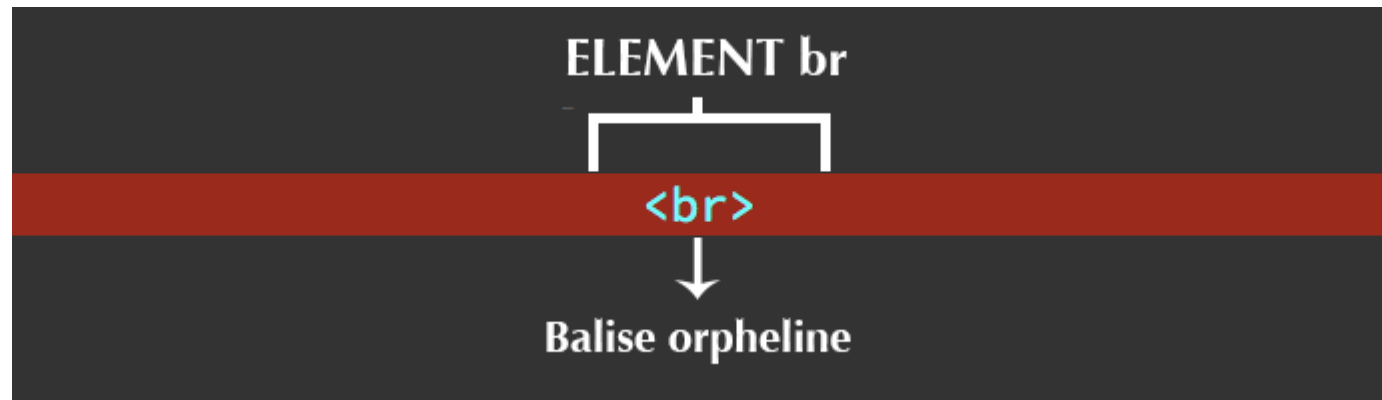
Un élément **HTML** peut être soit constitué d'une paire de balises (ouvrante et fermante) et d'un contenu, soit d'une balise unique qu'on dit alors « orpheline ».

- L'élément **p** (qui sert à définir un paragraphe) est par exemple constitué d'une balise ouvrante, d'une balise fermante et d'un contenu textuel entre les balises. L'idée ici est que le texte contenu entre les deux balises va être le texte considéré par le navigateur comme étant un paragraphe.
- Voici comment on va écrire cela :



Comme vous pouvez le constater, la balise ouvrante de l'élément est constituée d'un chevron ouvrant `<`, du nom de l'élément en question et d'un chevron fermant `>`.

- Notez bien ici la différence entre la balise ouvrante et la balise fermante de notre élément **p** : la balise fermante contient un slash avant le nom de l'élément.
- Vous pouvez déjà retenir cette syntaxe qui sera toujours la même en **HTML**.
- Certains éléments en **HTML** ne vont être constitués que d'une balise qu'on appelle alors orpheline. Cela va être le cas pour certains éléments qui ne possèdent pas de contenu textuel comme l'élément **br** par exemple qui sert simplement à créer un retour à la ligne en **HTML** et qui va s'écrire comme ceci :



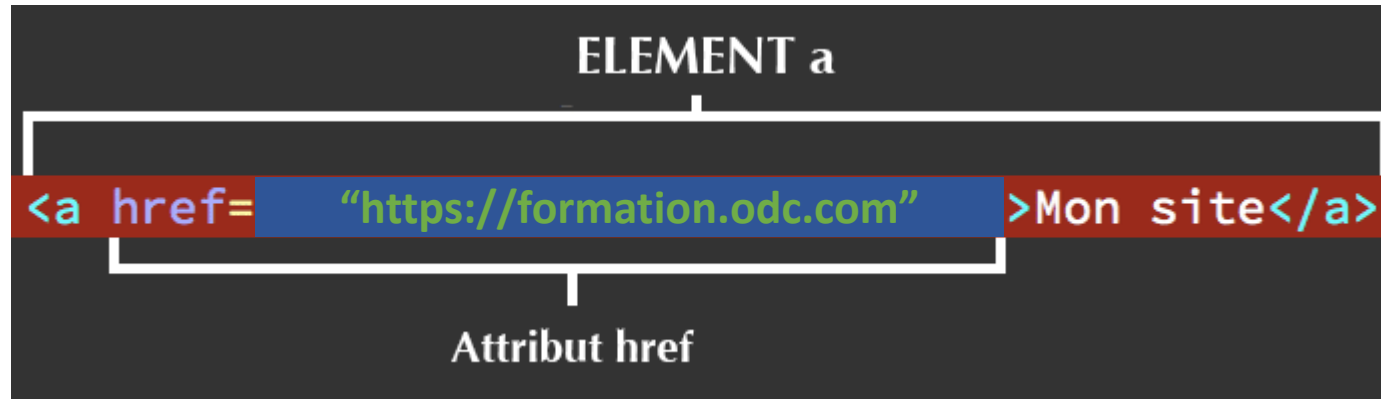
- Notez ici qu'il est possible que vous rencontriez un jour une syntaxe un peu différente pour les balises orphelines utilisant un slash après le nom de l'élément comme ceci : `<br/>`.
- Cette syntaxe est une syntaxe qui était acceptée il y a quelques années mais qui est aujourd'hui dépréciée en **HTML**. Elle provient en fait d'un autre langage qui est le **XML**. Il est déconseillé de l'utiliser aujourd'hui en **HTML** puisqu'elle risque de ne plus être reconnue par les navigateurs dans le futur.
- Par ailleurs, notez que certains développeurs ont tendance, par abus de langage, à confondre les termes « **élément** » et « **balise** » en utilisant le mot « **balise** » pour désigner un élément. Vous verrez peut-être cela écrit dans d'autres cours.
- C'est toutefois un abus de langage et je pense qu'il est préférable pour une bonne compréhension d'appeler chaque objet d'un langage par son vrai nom.



Les attributs HTML

- Finalement, les éléments vont également pouvoir contenir des attributs qu'on va alors placer au sein de la balise ouvrante de ceux-ci. Pour certains éléments, les attributs vont être facultatifs tandis que pour d'autres ils vont être obligatoires pour garantir le bon fonctionnement du code **HTML**.
- Les attributs vont en effet venir compléter les éléments en les définissant plus précisément ou en apportant des informations supplémentaires sur le comportement d'un élément.
- Un attribut contient toujours une valeur, qu'on peut cependant parfois omettre dans le cas des attributs ne possédant qu'une seule valeur (car la valeur est alors considérée comme évidente).
- Prenons ici l'exemple de l'élément **a** qui est l'abréviation de « **anchor** » ou « ancre » en français. Cet élément va principalement nous servir à créer des liens vers d'autres pages.
- Pour le faire fonctionner correctement, nous allons devoir lui ajouter un attribut **href** pour « **hypertexte reference** » ou « **référence hypertexte** » en français.

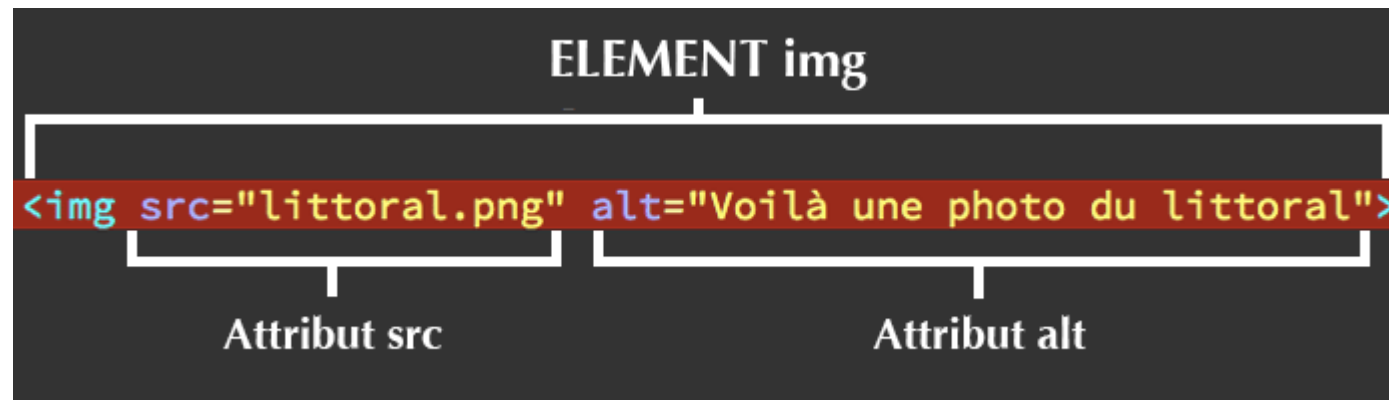
En effet, c'est l'attribut **href** qui va nous permettre de préciser la cible du lien.



Un autre exemple : l'élément **img**, servant à insérer une image dans une page **HTML**, va lui nécessiter deux attributs qui sont les attributs **src** et **alt**.

L'attribut **src** (pour « source ») va prendre comme valeur l'adresse de l'image tandis que l'attribut **alt** (pour « alternative ») va nous permettre de renseigner une description textuelle de l'image qui sera affichée dans les cas où l'image ne serait pas disponible pour une raison ou une autre : image introuvable, impossible à charger, etc.

L'attribut **alt** va également se révéler indispensable pour rendre notre site accessible aux non-voyants ou aux mal voyants et pour leur fournir une bonne expérience de navigation puisqu'ils sont généralement équipés de lecteur spéciaux qui vont pouvoir lire la valeur de l'attribut **alt** et donc leur permettre de se faire une représentation du contenu de l'image.



Nom de l'attribut

Valeur de l'attribut

Balise orpheline

Notez au passage que l'élément **img** n'est constitué que d'une seule balise orpheline, tout comme l'élément **br** vu précédemment. On place dans ce cas les attributs au sein de la balise orpheline.

L'imbrication des balises et des éléments

Un autre concept qu'il vous faut comprendre absolument pour coder en **HTML** est la façon dont les éléments **HTML** s'imbriquent les uns dans les autres.

L'imbrication est l'une des fonctionnalités du **HTML** qui fait toute sa force.

Par exemple, ici, **title** et **meta** sont deux éléments « enfants » de l'élément **head** (car ils sont contenus à l'intérieur de celui-ci), mais sont deux « frères » : aucun des deux ne contient l'autre.

L'indentation en HTML

Indenter correspond à créer des retraits en début de ligne dans votre éditeur de façon cohérente et logique.

Indenter va nous permettre d'avoir un code plus propre et plus lisible, donc plus compréhensible. Indenter permet également de plus facilement détecter les erreurs potentielles dans un code.

Le but de l'indentation est de nous permettre de discerner plus facilement les différents éléments ou parties de code.

Sans indentation:

```
<> index.html ●
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  <meta charset="UTF-8">
5  <title>ODC-GUINEE</title>
6  </head>
7  <body>
8  <!-- Un titre important non indenté -->
9
10 <h1>Mon titre principal mal indenté</h1>
11
12 <!-- Un paragraphe non indenté -->
13
14 <p>Je suis un paragraphe mal indenté</p>
15
16 </body>
17 </html>
```

Avec indentation :

```
<> index.html ●
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title>ODC-GUINEE</title>
6      </head>
7      <body>
8          <!-- Un titre important non indenté -->
9          <h1>Mon titre principal mal indenté</h1>
10         <!-- Un paragraphe non indenté -->
11         <p>Je suis un paragraphe mal indenté</p>
12     </body>
13 </html>
```

Les commentaires

1. Définition et utilité des commentaires en HTML

- Les commentaires vont être des lignes de texte que l'on va écrire au milieu de notre code, afin de donner des indications sur ce que fait le code en question.
- Les commentaires ne seront pas affichés par le navigateur lorsque celui-ci va afficher la page : ils ne vont servir qu'aux développeurs créant ou lisant le code.
- Les commentaires vont pouvoir être très utiles dans trois situations :
 - 1) Dans le cas d'un gros / long projet, afin de bien se rappeler soi-même pourquoi nous avons écrit tel ou tel code, ou encore pour se repérer dans le code ;
 - 2) Si l'on souhaite distribuer son code, ou si l'on travaille à plusieurs, cela fait beaucoup plus professionnel et permet aux autres développeurs de comprendre beaucoup plus rapidement et facilement le code distribué ;
 - 3) Pour « **neutraliser** » une certaine partie d'un code sans toutefois le supprimer. Il suffit en effet de placer toute la partie du code en question en commentaire afin que celui-ci soit ignoré par le navigateur.

Syntaxe et écriture des commentaires en **HTML**

- Les commentaires peuvent être mono-ligne (c'est-à-dire écrits sur une seule ligne dans l'éditeur) ou multi-lignes (écrits sur plusieurs lignes dans l'éditeur).
- A la différence de nombreux autres langages informatiques utilisant également des commentaires, la syntaxe utilisée pour créer un commentaire mono-ligne va être la même que celle pour créer un commentaire multi-lignes en **HTML**.
- Voici comment s'écrit un commentaire en **HTML** :



Commentaire en HTML

```
<> index.html X
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>ODC-GUINEE</title>
7  </head>
8  <body>
9      <!--Ceci est un commentaire mono-ligne-->
10
11      <h1>Bonjour ODC</h1>
12
13      <!-- Ceci est un commentaire
14      multi-lignes -->
15  </body>
16  </html>
```

Même si vous ne voyez pas forcément l'intérêt de commenter dès maintenant, je vous garantis que c'est souvent ce qui sépare un bon développeur d'un développeur moyen. Faites donc l'effort d'intégrer cette bonne pratique dès à présent, cela vous sera bénéfique dans le futur.

Les titres

1. Définition de titres en HTML

- Il existe six niveaux hiérarchiques de titres (“heading” en anglais) définis par les éléments **h1**, **h2**, **h3**, **h4**, **h5** et **h6** qui vont nous permettre d’organiser le contenu dans nos pages.
- Bon à savoir : « **h** » signifie « **heading** », soit l’équivalent du mot « **titre** » en français. Les éléments en HTML portent souvent l’initiale de ce qu’ils représentent, en anglais.
- L’élément **h1** représente un titre principal dans notre page ou dans une section de page et, à ce titre, nous n’allons pouvoir utiliser qu’un seul élément **h1** par page (ou par section).



Les titres en HTML

```
<> index.html X
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>ODC-GUINEE</title>
6  </head>
7  <body>
8
9      <!-- Un titre très important -->
10     <h1>Mon titre principal</h1>
11
12     <!-- Un titre important -->
13     <h2>Je suis un titre important</h2>
14
15     <!-- Un titre un peu moins important -->
16     <h3>Je suis un titre d'importance moyenne</h3>
17
18     <!-- Etc, etc. -->
19     <h4>Un titre pas très important</h4>
20     <h5>Un titre peu important</h5>
21     <h6>Un titre vraiment peu important</h6>
22 </body>
23 </html>
```

Résultat

Mon titre principal

Je suis un titre important

Je suis un titre d'importance moyenne

Un titre pas très important

Un titre peu important

Un titre vraiment peu important

Les paragraphes

1. Définition de paragraphe en HTML

- Pour créer des paragraphes en HTML, nous allons utiliser l'élément `p`.
- On peut créer autant de paragraphes que l'on souhaite dans une page. A chaque nouveau paragraphe, il faut utiliser un nouvel élément `p`.
- Pour chaque nouveau paragraphe, un retour à la ligne va être créé automatiquement et affiché par votre navigateur (exactement comme c'était le cas avec les titres)



<> index.html X



<> index.html > ...

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>ODC-GUINEE</title>
6  </head>
7  <body>
8      <!-- Un titre important -->
9      <h1>Mon titre principal</h1>
10
11     <!-- Deux paragraphes différents -->
12     <p>
13         Pour créer une page web il ne suffit pas de taper simplement du texte. En plus de ce texte, il
14         faut aussi écrire ce qu'on appelle des balises, qui vont donner des instructions à l'ordinateur comme « aller à
15         la ligne », « afficher une image », etc.
16     </p>
17     <p>
18         Bon, tout cela était trop facile. Évidemment, il a fallu que ces satanés informaticiens s'en mêlent et
19         compliquent les choses. Il ne
20         suffit pas d'écrire « simplement » du texte dans l'éditeur, il faut aussi donner des instructions à l'ordinateur.
21         En HTML, on utilise pour cela des balises.
22     </p>
23 </body>
</html>
```

Résultat

Mon titre principal

Pour créer une page web il ne suffit pas de taper simplement du texte. En plus de ce texte, il faut aussi écrire ce qu'on appelle des balises, qui vont donner des instructions à l'ordinateur comme « aller à la ligne », « afficher une image », etc.

Bon, tout cela était trop facile. Évidemment, il a fallu que ces satanés informaticiens s'en mêlent et compliquent les choses. Il ne suffit pas d'écrire « simplement » du texte dans l'éditeur, il faut aussi donner des instructions à l'ordinateur. En HTML, on utilise pour cela des balises.

Le lien

1. Définition d'un lien **HTML**

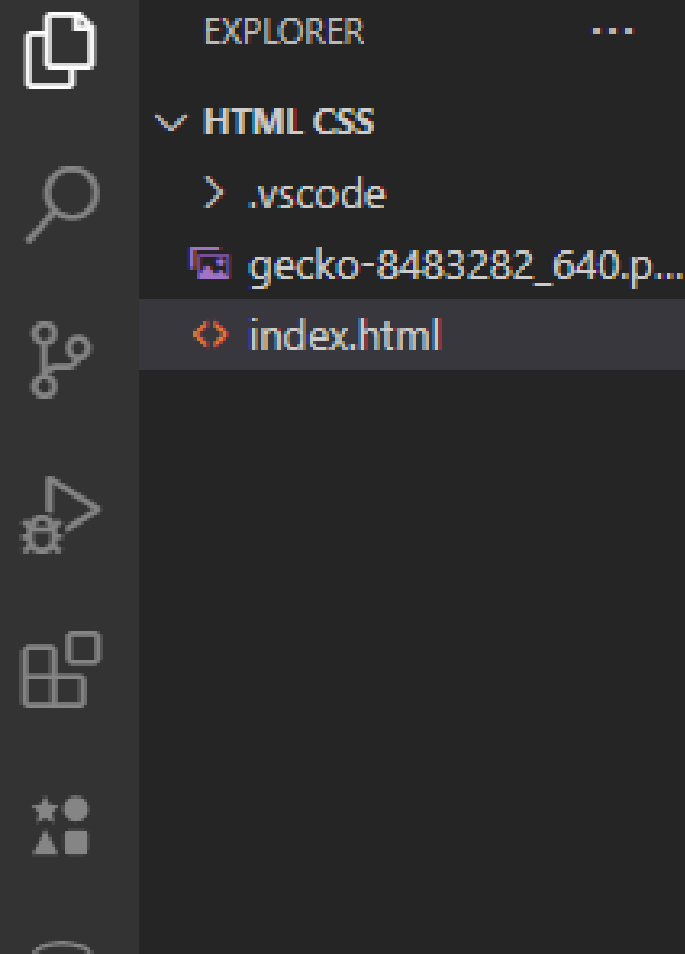
- Les liens en **HTML** vont nous servir à créer des ponts entre différentes pages d'un même site ou de sites différents. Le principe d'un lien est le suivant : en cliquant sur une ancre (qui est la partie visible par l'utilisateur d'un lien et qui peut être un texte comme une image), nos utilisateurs vont être redirigés vers une page cible.
- Il existe deux types principaux de liens hypertextes en **HTML** :
- Les liens internes qui vont servir à naviguer d'une page à l'autre dans un même site ;
- Les liens externes qui vont envoyer les utilisateurs vers une page d'un autre site.
- Dans chacun de ces cas, nous allons devoir utiliser l'élément **HTML** `a` accompagné de son attribut `href` pour créer un lien en **HTML**.

- On peut utiliser l'élément **a** pour transmettre notre adresse mail à nos utilisateurs et leur permettre de nous envoyer simplement un mail.
- Pour permettre l'envoi d'un mail en **HTML**, on va placer indiquer en valeur de l'attribut **href** de notre élément a la valeur **mailto** : suivie de notre adresse email.
- On peut encore utiliser l'élément **a** pour permettre à vos visiteurs de télécharger certains types de fichiers, comme des fichiers PDF ou Word par exemple.

Les images

1. Définition d'images en **HTML**

- La balise qui permet d'insérer une image est une balise orpheline : ****.
- Pour fonctionner correctement, la balise **** doit être accompagnée de deux attributs :
- **src** : cet attribut permet d'indiquer la source de l'image.
- **alt** : cet attribut permet de donner à l'image une description alternative.



<> index.html X

<> index.html > ...

```
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title>ODC-GUINEE</title>
6      </head>
7      <body>
8          <!-- Définition d'un titre en HTML -->
9          <h2>Page web ODC-GUINEE</h2>
10         <!-- Définir une image en HTML -->
11         
12     </body>
13 </html>
```

Résultat

Page web ODC-GUINEE



Les listes

1. Les types de listes en HTML

Il existe deux grands types de listes en HTML : les listes ordonnées et les listes non-ordonnées. Il existe également un troisième type de liste un peu particulier et moins utilisé : les listes de définitions.

Les listes non-ordonnées

- Les listes non-ordonnées vont être utiles pour lister des éléments sans hiérarchie ni ordre logique.
- Par exemple, si je souhaite lister les mots « pomme », « vélo » et « guitare », sans plus de contexte, j'utiliserai une liste non-ordonnée.
- En effet, on ne peut pas dégager de notion d'ordre, de hiérarchie ou de subordination entre ces trois termes (du moins pas sans un contexte précis).

Pour créer une liste non-ordonnée, nous allons avoir besoin d'un élément **ul** (pour « **unordered list** », ou « liste non-ordonnée » en français) qui va représenter la liste en soi ainsi que d'un élément **li** (« **list item** » ou « **élément de liste** ») pour chaque nouvel élément de liste.

```
<> index.html X
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title>ODC-GUINEE</title>
6      </head>
7      <body>
8          <!-- Définition de liste -->
9          <h1>Les listes</h1>
10         <!-- Un exemple de liste non-ordonnée -->
11         <ul>
12             <li>Légumes</li>
13             <li>Fruits</li>
14             <li>Viandes</li>
15             <li>Poissons</li>
16         </ul>
17
18     </body>
19 </html>
```

Résultat

Les listes

- Légumes
- Fruits
- Viandes
- Poissons

Les listes ordonnées

- Au contraire des listes non-ordonnées, nous allons utiliser les listes ordonnées lorsqu'il y aura une notion d'ordre ou de progression logique ou encore de hiérarchie entre les éléments de notre liste.
- Par exemple, si l'on souhaite lister les étapes naturelles de la vie, ou lorsque l'on crée un sommaire, on utilisera généralement des listes ordonnées.
- Pour créer une liste ordonnée, nous allons cette fois-ci utiliser l'élément `ol` (pour « ordered list » ou « liste ordonnée ») pour définir la liste en soi et à nouveau des éléments `li` pour chaque élément de la liste.



Listes ordonnées

```
<> index.html X
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  <meta charset="UTF-8">
5  <title>ODC-GUINEE</title>
6  </head>
7  <body>
8  <!-- Définition de liste -->
9  <h1>Les listes</h1>
10 <!-- Deux exemples de listes ordonnées -->
11 <p>Liste n°1</p>
12 <ol>
13 <li>Primaire</li>
14 <li>College</li>
15 <li>Lycée</li>
16 <li>Université</li>
17 </ol>
18 <p>Liste n°2</p>
19 <ol>
20 <li>Lundi</li>
21 <li>Mardi</li>
22 <li>Mercredi</li>
23 <li>Jeudi</li>
24 <li>Vendredi</li>
25 </ol>
26
27 </body>
28 </html>
```

Résultat

Les listes

Liste n°1

1. Primaire
2. Collège
3. Lycée
4. Université

Liste n°2

1. Lundi
2. Mardi
3. Mercredi
4. Jeudi
5. Vendredi

Les listes de définitions

- Les listes de définitions, encore appelées « listes de descriptions » vont nous permettre de lister des termes et d'ajouter des définitions ou descriptions pour chacun de ces termes.
- Pour créer une liste de définitions, nous allons cette fois-ci utiliser l'élément **dl** signifiant « description list » ou « liste de description / définition » en français pour définir la liste en soi, puis des éléments **dt** (description term) pour chaque élément à décrire et enfin l'élément **dd** pour la définition / description en soi.
- Pensez bien lorsque vous créez une liste de définitions à toujours placer le terme à définir avant sa définition, c'est-à-dire l'élément **dt** avant l'élément **dd**. Cela est normalement assez intuitif.



Listes de définitions

<> index.html X

<> index.html > ...

```
1  <!DOCTYPE html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8">
5      <title>ODC-GUINEE</title>
6    </head>
7    <body>
8      <!-- Définition de liste -->
9      <h1>Les listes</h1>
10     <!-- Une liste de définition -->
11     <dl>
12       <dt>HTML</dt>
13       <dd>HTML signifie HyperText Markup Language.</dd>
14     </dl>
15     </ol>
16
17   </body>
18 </html>
```

Résultat

Les listes

HTML

THML signifie HyperText Markup Language.

Les entités HTML

- Le **HTML** possède des caractères réservés. Par exemple, vous ne pouvez pas écrire les signes « **<** » et « **>** » tels quels dans vos pages web, tout simplement car le navigateur pensera que vous venez d'ouvrir une balise d'élément.
- Pour remédier à ce problème, nous allons devoir « échapper » ces caractères réservés en utilisant ce qu'on appelle des entités **HTML**. Les entités vont être des suites de caractères représentant un caractère réservé en **HTML**.
- Voici quelques-unes des entités les plus courantes et leur signification :

Nom de l'entité	Résultat visuel
<	< (chevron ouvrant)
>	> (chevron fermant)
&	& (esperluette)
 	(espace insécable)

Les formulaires HTML

- Pour créer un formulaire, nous allons utiliser l'élément HTML **form**. Cet élément **form** va avoir besoin de deux attributs pour fonctionner normalement : les attributs **method** et **action**.
- L'attribut **method** va indiquer comment doivent être envoyées les données saisies par l'utilisateur. Cet attribut peut prendre deux valeurs : **get** et **post**.
- Que signifient ces deux valeurs et laquelle choisir ? Les valeurs **get** et **post** vont déterminer la méthode de transit des données du formulaire. En choisissant **get**, on indique que les données doivent transiter via l'URL (sous forme de paramètres) tandis qu'en choisissant la valeur **post** on indique qu'on veut envoyer les données du formulaire via transaction post **HTTP**.
- Si l'on choisit l'envoi via l'URL (avec la valeur **get**), nous serons limités dans la quantité de données pouvant être envoyées et surtout les données vont être envoyées en clair. Evitez donc absolument d'utiliser cette méthode si vous demandez des mots de passe ou toute information sensible dans votre formulaire.

- En choisissant l'envoi de données via **post** transaction **HTTP** (avec la valeur **post**), nous ne sommes plus limités dans la quantité de données pouvant être envoyées et les données ne sont visibles par personne. C'est donc généralement la méthode que nous utiliserons pour l'envoi véritablement de données que l'on souhaite stocker (création d'un compte client, etc.).
- L'attribut **action** va lui nous servir à préciser l'adresse de la page qui va traiter les données.

Les éléments de formulaires HTML

La liste des éléments que l'on va pouvoir utiliser dans nos formulaires **HTML** ainsi que leur rôle :



Élément	Définition
form	Définit un formulaire
input	Définit un champ de données pour l'utilisateur
label	Définit une légende pour un élément input
textarea	Définit un champ de texte long
select	Définit une liste de choix
optgroup	Définit un groupe d'options dans une liste
option	Définit une option dans une liste
fieldset	Permet de regrouper les éléments d'un formulaire en différentes parties
legend	Ajoute une légende à un élément fieldset

L'élément input et les valeurs de l'attribut type

- L'élément **input** est l'élément à connaître dans le contexte de la création de formulaires **HTML** puisqu'il va nous permettre de créer tous types de champs pour récolter des données utilisateurs.
- Le type de champ va être défini par la valeur que l'on va donner à son attribut **type**. L'attribut **type** peut prendre de nombreuses valeurs. Voici les valeurs les plus utiles et les plus courantes



Type d'input	Définition
text	Définit un champ de saisie monoligne qui accepte du texte
number	Définit un champ qui accepte un nombre décimal
email	Crée un champ qui permet de renseigner une adresse mail
date	Permet à l'utilisateur d'envoyer une date
password	Créer un champ de saisie monoligne dont la valeur va être cachée
checkbox	Permet de créer une case à cocher. L'utilisateur peut cocher une ou plusieurs cases d'un coup
radio	Permet de créer un bouton radio. Par définition, un seul bouton radio peut être coché dans un ensemble
url	Crée un champ qui accepte une URL
file	Permet à un utilisateur de télécharger un fichier
submit	Crée un bouton d'envoi des données du formulaire

Les éléments fieldset et legend

- Les éléments **fieldset** et **legend** vont nous permettre de structurer et d'améliorer la sémantique d'un formulaire.
- L'élément **fieldset** va nous permettre de grouper plusieurs champs dans un formulaire pour l'organiser en parties.
- L'élément **legend** va nous permettre d'ajouter une légende à une partie de notre formulaire déterminée par **fieldset** pour expliquer son but par exemple.



JOUR 2: CSS - MISE EN FORME DES PAGES WEB

Introduction au CSS:

- **CSS** signifie Cascading Style Sheets (feuilles de style en cascade). C'est un langage informatique utilisé pour styliser les pages web. Il permet de contrôler l'apparence des éléments **HTML**, tels que les titres, les paragraphes, les images, les liens hypertextes, etc.

Comment fonctionne CSS ?

- Le code CSS est écrit dans un fichier distinct du code **HTML**.
- Le fichier CSS est lié à la page HTML à l'aide d'une balise `<link>`
- Les navigateurs web interprètent le code CSS et appliquent les styles aux éléments **HTML** correspondants.

Propriétés CSS

Le langage **CSS** utilise des propriétés pour définir l'apparence des éléments. Chaque propriété a un **nom** et une **valeur**. Par exemple, la propriété **color** permet de définir la couleur du texte d'un paragraphe.

Sélecteurs CSS

Les sélecteurs **CSS** permettent de cibler les éléments **HTML** auxquels on souhaite appliquer des styles. Il existe différents types de sélecteurs, comme les sélecteurs par élément, les sélecteurs par classe, les sélecteurs par **ID**, etc.

Exemple de code **CSS**



La valeur de la propriété CSS :

- Définition de la valeur de la propriété CSS

La valeur de la propriété CSS est l'information qui détermine l'effet de la propriété sur l'élément ciblé. Elle peut être exprimée de différentes manières, selon la propriété en question.

- Types de valeurs de propriété CSS

On distingue plusieurs types de valeurs de propriété CSS :

1. **Valeurs littérales:** Elles sont définies directement dans le code CSS, comme "blue" pour la couleur ou "24px" pour la taille de la police
2. **Valeurs fonctionnelles:** Elles font appel à des fonctions CSS pour générer la valeur, comme `rgb(0, 0, 255)` pour définir une couleur bleue.
3. **Valeurs calculées:** Elles sont déterminées par le navigateur en fonction de divers facteurs, comme la taille de l'écran ou la police par défaut du système.
4. **Valeurs relatives:** Elles sont exprimées par rapport à d'autres valeurs, comme des pourcentages ou des unités relatives (`em`, `rem`).

Où écrire le code CSS ?

Avant d'étudier les mécanismes du **CSS** en soi, il convient de comprendre où placer le code **CSS** afin qu'il s'applique bien à un fichier **HTML**.

Pour faire cela, nous allons pouvoir écrire le code **CSS** à trois endroits différents. Chaque méthode va présenter des avantages et des inconvénients selon une situation donnée.

Méthode n°1 : écrire le **CSS au sein du fichier **HTML**, dans un élément style**

- La première façon d'écrire du code **CSS** va être à l'intérieur même de notre page **HTML**, au sein d'un élément style.
- En plaçant le **CSS** de cette façon, le code **CSS** ne s'appliquera qu'aux éléments de la page **HTML** dans laquelle il a été écrit.

Cette première méthode d'écriture du **CSS** n'est pas recommandée, pour des raisons de maintenance et d'organisation du code en général. Cependant, elle peut s'avérer utile pour modifier rapidement les styles d'une page **HTML** ou si vous n'avez pas facilement accès aux fichiers de style de votre site.

Nous voyons donc cette première méthode à titre d'exemple, afin que vous sachiez l'identifier si un jour vous voyez du code CSS écrit de cette façon dans un fichier et que vous puissiez l'utiliser si vous n'avez pas d'autre choix.



Nous allons devoir ici placer notre élément **style** au sein de l'élément **head** de notre fichier **HTML**.

Voici comment on va écrire cela en pratique :

```
index.html X
index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title>ODC-GUINEE</title>
6          <style>
7              body{
8                  background-color: grey;
9              }
10             p{
11                 color: blue;
12                 font-size: 16px;
13             }
14         </style>
15     </head>
16     <body>
17         <h1>Un titre de niveau 1</h1>
18         <p>Un paragraphe</p>
19         <p>Un deuxième paragraphe</p>
20     </body>
21 </html>
```

Résultat

Un titre de niveau 1

Un paragraphe

Un deuxième paragraphe

Méthode n°2 : déclarer le CSS au sein du fichier HTML, dans des attributs style

- Nous pouvons également écrire notre code CSS au sein d'attributs style qu'on va ajouter à l'intérieur de la balise ouvrante des éléments HTML pour lesquels on souhaite modifier les styles.
- Nous allons passer en valeurs des attributs style des déclarations CSS pour modifier certains styles précis de l'élément HTML en question. En effet, en utilisant cette méthode, les styles déclarés dans un attribut style ne vont s'appliquer qu'à l'élément dans lequel ils sont écrits, et c'est la raison pour laquelle nous n'allons pas avoir besoin de préciser de sélecteur ici.
- Attention à ne pas confondre les attributs style qu'on va devoir placer au sein de la balise ouvrante de chaque élément dont on souhaite modifier les styles avec l'élément style qu'on va placer dans l'élément head de nos fichiers HTML.

Voici comment on va écrire cela en pratique :

index.html X

index.html > ...

```
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title>ODC-GUINEE</title>
6      </head>
7      <body style="background-color: grey;">
8          <h1>Un titre de niveau 1</h1>
9          <p style="color: blue; font-size: 20px;">Un paragraphe</p>
10         <p>Un deuxième paragraphe</p>
11     </body>
12 </html>
```

Résultat

Un titre de niveau 1

Un paragraphe

Un deuxième paragraphe

- Cette deuxième méthode d'écriture du **CSS**, bien qu'elle puisse sembler pratique à priori puisqu'elle permet de n'appliquer des styles qu'à un élément en particulier plutôt qu'à tous les éléments d'un même type n'est également pas recommandée et est à éviter tant que possible pour des raisons de maintenabilité et de performance du code.
- En effet, déclarer nos styles comme cela n'est vraiment pas efficient puisque cela va demander énormément d'écriture et également énormément de temps de réécriture le jour où l'on souhaite modifier des styles.



Méthode n°3 : écrire le CSS dans un fichier séparé

- Finalement, nous pouvons écrire notre code **CSS** dans un fichier séparé portant l'extension « **.css** ». C'est la méthode recommandée, qui sera utilisée autant que possible.
- Cette méthode comporte de nombreux avantages, notamment une meilleure maintenabilité du code grâce à la séparation des différents langages, ainsi qu'une meilleure lisibilité.
- Cependant, le plus gros avantage de cette méthode est qu'on va pouvoir appliquer des styles à plusieurs pages **HTML** en même temps, d'un seul coup.
- En effet, en utilisant l'une des deux premières méthodes, nous aurions été obligés de réécrire tout notre code **CSS** pour chaque page **HTML** (ou même pour chaque élément !) composant notre site puisque les codes **CSS** étaient déclarés dans une page ou dans un élément spécifique et ne pouvaient donc s'appliquer qu'à la page ou qu'à l'élément dans lesquels ils étaient déclarés.

En déclarant notre code **CSS** dans un fichier séparé, au contraire, nous allons pouvoir utiliser le code de ce fichier **CSS** dans autant de fichiers **HTML** qu'on le souhaite, en indiquant aux différents fichiers **HTML** qu'ils doivent appliquer les styles contenus dans ce fichier **CSS**. Ainsi, lorsque nous voudrions modifier par exemple la couleur de tous les paragraphes de nos pages **HTML** nous n'aurons qu'à modifier la déclaration relative dans le fichier **CSS**.

Nous travaillons donc dorénavant avec deux fichiers : un fichier appelé **index.html** et un fichier **style.css**.

Nous allons utiliser un nouvel élément **HTML** : l'élément **link** (« lien », en français). On va placer l'élément **link** au sein de l'élément **head** de notre fichier **HTML**. Cet élément se présente sous la forme d'une balise orpheline et va avoir besoin de deux attributs pour fonctionner correctement :

1. Un attribut **rel** qui va nous servir à préciser le type de ressource que l'on souhaite lier à notre fichier **HTML**. Dans notre cas, nous indiquerons la valeur **stylesheet** pour « feuille de style » ;

1. Un attribut **rel** qui va nous servir à préciser le type de ressource que l'on souhaite lier à notre fichier **HTML**. Dans notre cas, nous indiquerons la valeur stylesheet pour « **feuille de style** » ;
2. Un attribut **href** qui va indiquer l'adresse relative de la ressource que l'on souhaite lier par rapport à l'emplacement de notre fichier **HTML**.

Voici comment on va écrire cela en pratique :

```
4      <head>
5      <meta charset="UTF-8">
6      <title>ODC-GUINEE</title>
7      <link rel="stylesheet" href="style.css">
8      </head>
```

Voici un extrait du code **css**

```
<> index.html # style.css •
# style.css > ...
1  p{
2      color: blue;
3      border: 2px solid orange;
4      padding: 5px;
5      margin-right: 70%;
6  }
```

Résultat

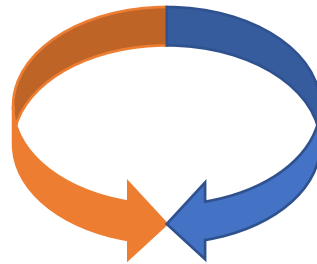
Un titre de niveau 1

Un paragraphe

Un deuxième paragraphe

• Commenter le code CSS

- Tout comme nous avons vu qu'on pouvait écrire des commentaires en **HTML**, nous allons également pouvoir commenter notre code **CSS**.
- Le **CSS**, tout comme le **HTML** et à la différence d'autres langages de développement ne possède qu'une seule syntaxe qui va nous permettre de créer à la fois des commentaires mono-ligne et multi-lignes.
- Cette syntaxe est la suivante :



Voici comment on va écrire cela en pratique :

```
<> index.html # style.css ●
# style.css > ...
1  /* Un premier commentaire en CSS */
2  ∨ body{
3    background-color: grey;
4  }
5
6  ∨ /* Un deuxième commentaire
7    sur plusieurs
8    lignes */
9
10 ∨ p{
11   /* color: blue; */
12   font-size: 18px;
13 }
14
```

2 - Mise en forme du texte:

- Les propriétés **CSS** liées au texte : Dans cette partie, nous allons nous intéresser aux propriétés **CSS** permettant de mettre en forme des textes.
- Les propriétés **CSS** liées au texte peuvent être séparées en deux grandes catégories :
 1. Les propriétés de type **font-** qui vont définir l'aspect des caractères en soi en qui agissent directement sur la police d'écriture (choix de la police, de la taille des caractères ou de leur poids entre autres);
 2. Les propriétés de type **text-** qui ne vont pas impacter directement l'aspect des caractères du texte mais nous permettre d'ajouter des effets de style autour de celui-ci (alignement du texte, soulignement ou encore ajout d'ombres autour des textes par exemple).

Voici quelques-unes des propriétés **CSS** appliquées aux textes :

- La propriété **text-align** qui va nous permettre de gérer l'alignement du texte;
- La propriété **text-transform** qui va nous permettre de gérer la casse du texte (le fait que le texte soit en majuscules ou en minuscules);
- La propriété **text-decoration** qui va nous permettre d'ajouter des éléments de décoration autour du texte comme un trait de soulignement par exemple;
- La propriété **text-shadow** qui va nous permettre d'ajouter des ombres autour d'un texte;
- La propriété **letter-spacing** va nous permettre de définir l'espace entre les différents caractères de notre police;
- La propriété **font-family** va nous permettre de définir la police de nos textes, c'est-à-dire le rendu graphique de chaque caractère.
- La propriété **font-size** qui va nous permettre de modifier la taille de notre police;
- La propriété **font-weight** qui va nous permettre de modifier le poids de notre police;
- La propriété **font-style** qui va nous permettre de modifier l'inclinaison de notre police.

3 - Mise en forme des liens

CSS offre plusieurs options pour personnaliser l'apparence des liens hypertextes dans vos pages web. Vous pouvez modifier la couleur, la police, la décoration, le comportement au survol et bien plus encore.

1. Modifier la couleur des liens:

Par défaut, les liens hypertextes sont affichés en bleu dans la plupart des navigateurs. Vous pouvez modifier cette couleur en utilisant la propriété **color** :

2. Modifier la police des liens:

Vous pouvez également définir une police spécifique pour les liens en utilisant la propriété **font-family** :

3. Supprimer le soulignement des liens:

Le soulignement des liens est un comportement par défaut des navigateurs. Vous pouvez le supprimer avec la propriété **text-decoration** :

4. Ajouter une décoration aux liens:

En plus du soulignement, vous pouvez ajouter d'autres décorations aux liens, comme un trait de soulignement, une barre oblique ou une surbrillance avec la propriété **text-decoration** :

5. Modifier l'apparence des liens au survol:

Lorsque la souris passe sur un lien, son apparence peut changer. Vous pouvez définir les styles pour cet état avec le pseudo-sélecteur **:hover** :

6. Modifier l'apparence des liens visités:

Les liens visités, c'est-à-dire les liens sur lesquels l'utilisateur a déjà cliqué, peuvent avoir une apparence différente des liens non visités. Vous pouvez définir des styles spécifiques pour les liens visités avec le pseudo-sélecteur **:visited** :

7. Modifier l'apparence des liens actifs:

Un lien actif est celui sur lequel l'utilisateur est en train de cliquer. Vous pouvez définir des styles pour cet état avec le pseudo-sélecteur **:active** :

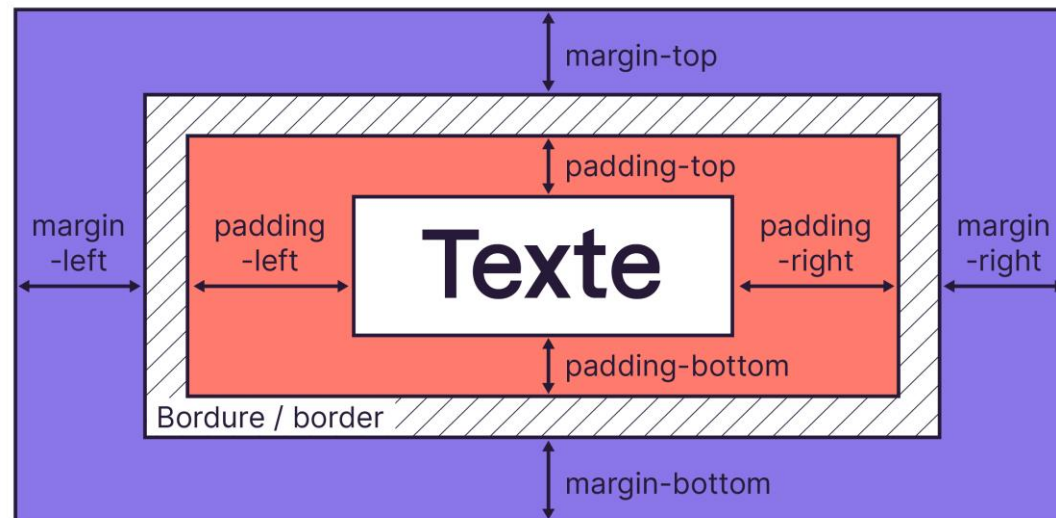
4 - Organisation des éléments:

Le modèle en boîte :

Le modèle en boîte est un ensemble de règles qui définissent l’affichage de chaque page Web sur Internet. **CSS** traite chaque élément de votre document **HTML** comme une «boîte» , un “box” avec un tas de propriétés différentes qui déterminent où ce “box” apparaît sur la page.

L’idée centrale du modèle en boîte est que tout élément **HTML** peut être représenté par un empilement de différentes boîtes rectangulaires :

Représentation d’un élément selon le modèle en boîte



1. La première boîte, centrale, va être composée du contenu de l'élément en soi;
2. La deuxième boîte va être composée de la première boîte ainsi que des marges internes (**padding**) de l'élément;
3. La troisième boîte va être composée de la deuxième boîte et des bordures de l'élément;
4. La quatrième boîte va être composée de la troisième boîte et des marges externes (**margin**) de l'élément.

- **Les propriétés CSS liées aux différentes boîtes**

Le **CSS** va déjà nous fournir différentes propriétés qui vont nous permettre de spécifier la taille des différents éléments composant les différentes boîtes :

1. Les propriétés **width** et **height** vont nous permettre de définir la largeur et la hauteur de la boîte «contenu»;
2. La propriété **padding** va nous permettre de définir la taille des marges internes;
3. La propriété **border** va nous permettre de définir des bordures pour notre élément;
4. La propriété **margin** va nous permettre de définir la taille des marges externes.

Les attributs HTML class et id et les sélecteurs CSS associés

- Les attributs **HTML class** et **id** sont des attributs dits globaux car on va pouvoir les ajouter dans la balise ouvrante de n'importe quel élément **HTML**.
- Ces deux attributs vont être principalement utilisés dans un but de mise en forme : ils vont nous servir à appliquer des styles **CSS** aux éléments qui vont les contenir.
- En effet, à la différence d'un attribut **href** par exemple, les attributs **class** et **id** ne vont pas servir à préciser le fonctionnement d'un élément **HTML** mais vont simplement être très utiles pour cibler un élément précisément.
- Nous allons effectivement très facilement pouvoir nous resservir de ces deux attributs en **CSS** grâce aux sélecteurs associés **.class** et **#id**.
- Notez déjà que chaque **id** doit avoir une valeur propre ou unique dans une page. En revanche, on va pouvoir attribuer la même valeur à plusieurs attributs **class** différents.

Code HTML:

```
<> index.html X # style.css
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title>ODC-GUINEE</title>
6          <link rel="stylesheet" href="style.css">
7      </head>
8      <body>
9          <h1 id="orange">Un titre de niveau 1</h1>
10         <p class="bleu">Un premier paragraphe</p>
11         <p class="bleu">Un autre paragraphe avec un <a href="#">lien</a></p>
12         <ul>
13             <li>Hommes</li>
14             <li class="vert">Femmes</li>
15         </ul>
16         <p id="gros">Un troisième paragraphe</p>
17     </body>
18 </html>
```

Code CSS:

```
<> index.html # style.css X
# style.css > ...
1  #orange{
2      color: orange;
3  }
4
5  #gros{
6      font-size: 25px;
7  }
8
9  .bleu{
10     color: blue;
11 }
12
13 .vert{
14     color: green;
15 }
```

Résultat

Un titre de niveau 1

Un premier paragraphe

Un autre paragraphe avec un [lien](#)

- Hommes
- Femmes

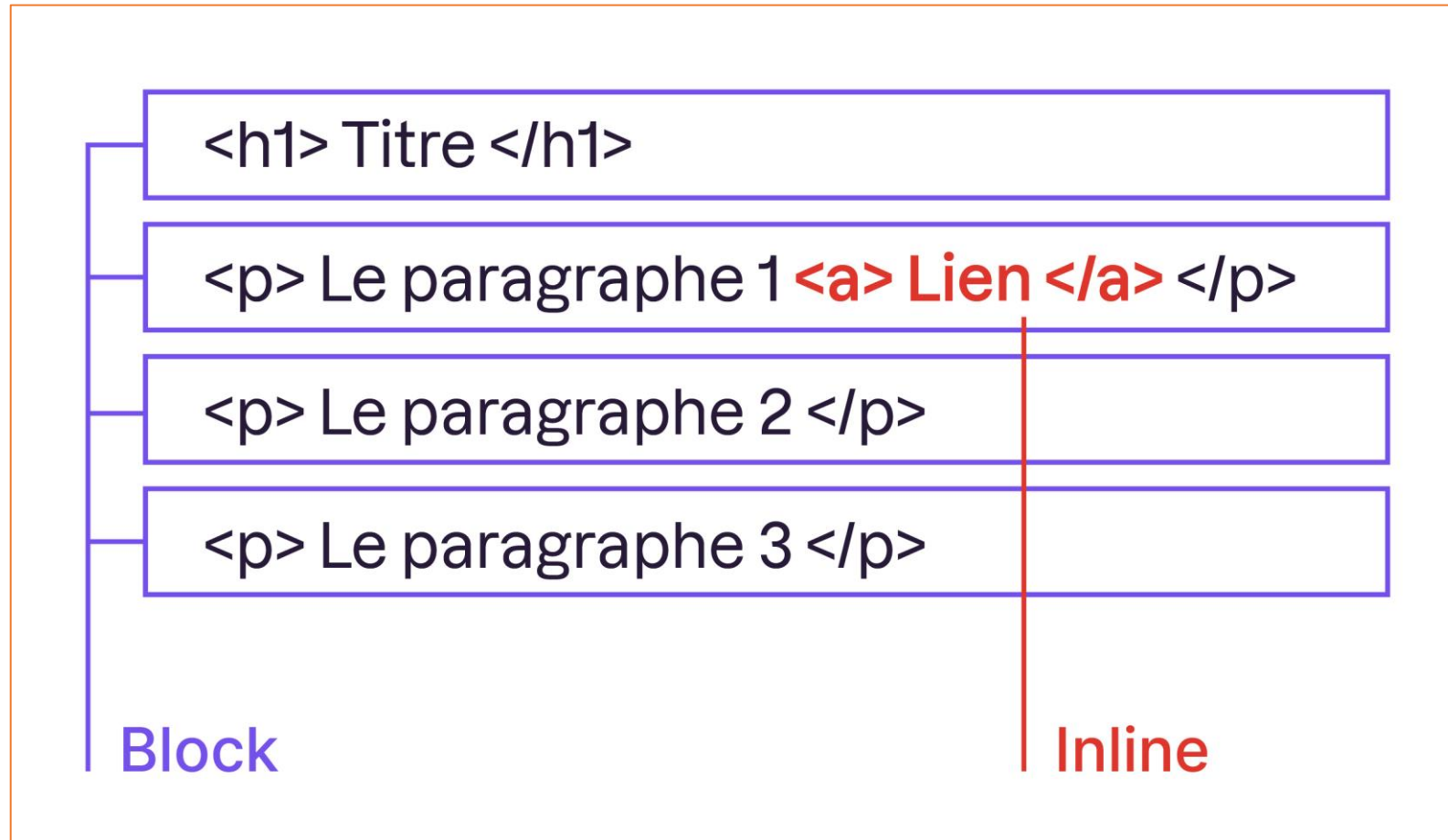
Un troisième paragraphe

Les balises de types block et inline

En **HTML** on a deux familles de balises :

- Les balises de type **block** : une balise de type block sur votre page web crée automatiquement un retour à la ligne avant et après et occupe toute la largeur disponible.
- Les balises **inlines** : une balise de type **inline** se trouve obligatoirement à l'intérieur d'une balise **block**. Une balise **inline** ne crée pas de retour à la ligne, le texte qui se trouve à l'intérieur s'écrit donc à la suite du texte précédent, sur la même ligne (c'est pour cela que l'on parle de balise «en ligne»).
- Connaître le type d'affichage d'un élément **HTML** va donc être essentiel pour créer et mettre en forme nos pages web car les éléments de type **block** et ceux de type **inline** vont se comporter de façon radicalement différente dans une page et certaines propriétés **CSS** vont avoir des comportements différents selon le type d'affichage d'un élément.

Pour bien visualiser le concept, voici un petit schéma :



Les éléments HTML de type inline les plus courants sont les suivants :

- Les éléments de distinction d'importance du contenu `em` et `strong` ;
- L'élément `span` ;
- L'élément de liens `a` ;
- L'élément `button` ;
- Les éléments de formulaire `input`, `label`, `textarea` et de liste de choix `select` ;
- L'élément d'insertion d'images `img` (cas intéressant et souvent source de confusions car on va pouvoir passer une largeur et une hauteur à l'image à afficher en soi qui va « remplacer » l'élément `img` lors de l'affichage, mais il n'empêche que l'élément `img` est bien `inline` en soi);
- Les éléments `code`, `script`, etc.

Code HTML:

```
<> index.html X # style.css
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title>ODC-GUINEE</title>
6          <link rel="stylesheet" href="style.css">
7      </head>
8      <body>
9          <h1>Un titre de niveau 1</h1>
10         <p>L'élément <mark>p</mark> est un élément de type block.</p>
11         <p>L'élément <strong>strong</strong> et l'élément <span>span</span> sont tous de type inline.</p>
12     </body>
13 </html>
```

Code CSS:

```
<> index.html # style.css
# style.css > ...
1  strong, span{
2      background-color: orange;
3  }
4
```

Résultat

Un titre de niveau 1

L'élément **p** est un élément de type block.

L'élément **strong** et l'élément **span** sont tous de type inline.

Les éléments HTML de type block les plus communs sont les suivants :

- L'élément **body**, cas particulier mais qui est concrètement considéré comme un élément **block** ;
- L'élément de division du contenu **div** ;
- Les paragraphes **p** et titres **h1**, **h2**, **h3**, **h4**, **h5**, **h6** ;
- Les éléments structurants **article**, **aside**, **header**, **footer**, **nav** et **section** ;
- Les listes **ul**, **ol**, **dl** et éléments de listes de définition **dt** et **dd** ;
- L'élément de définition de tableaux **table** ;
- L'élément de définition de formulaires **form** et l'élément **fieldset** ;
- Les éléments **figure** et **figcaption** ;
- Les éléments **canvas**, **video**, etc.

Exemples pratiques :

Code HTML:

```
<> index.html X # style.css
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title>ODC-GUINEE</title>
6          <link rel="stylesheet" href="style.css">
7      </head>
8      <body>
9          <h1>Un titre de niveau 1</h1>
10         <div class="conteneur">
11             <p>L'élément <mark>p</mark> est un élément de type block.</p>
12             <p>L'élément <strong>strong</strong> et l'élément <span>span</span> sont tous de type inline.</p>
13         </div>
14     </body>
15 </html>
```

Code CSS:

```
<> index.html # style.css
# style.css > ...
1  .conteneur{
2      border: 1px solid black;
3      background-color: lightgrey;
4      width: 50%;
5  }
6
7  p{
8      width: 100%;
9      background-color: aqua;
10 }
11
12 strong, span{
13     background-color: orange;
14 }
```

Résultat

Un titre de niveau 1

L'élément **p** est un élément de type block.

L'élément **strong** et l'élément **span** sont tous de type inline.

Les éléments HTML div et span :

Les éléments HTML **div** et **span** ont été créés principalement pour simplifier la mise en page de nos pages **HTML** en **CSS** c'est-à-dire pour simplifier l'application de certains styles **CSS**

Exercices:

- Mise en forme d'une page web avec CSS
- Application de styles aux titres, paragraphes et liens
- Création d'une mise en page attractive

JOUR 3: LA MAITRISE DU CSS

1 –Les sélecteurs CSS complexes et combinaisons

- Toute la puissance du **CSS** réside dans les options que nous offre ce langage pour cibler précisément un contenu **HTML** dans une page.
- En effet, en plus des sélecteurs simples, le **CSS** met à notre disposition une panoplie de sélecteurs que l'on va pouvoir utiliser pour cibler des contenus de manière très précise :
 1. Sélectionner tous les éléments avec le sélecteur CSS universel ou sélecteur étoile (*)
 2. Appliquer des styles à plusieurs éléments avec le caractère virgule (,)
 3. Utiliser plusieurs sélecteurs CSS à la suite
- En mentionnant plusieurs sélecteurs à la suite en **CSS**, nous allons pouvoir appliquer des styles à certains éléments contenus dans d'autres éléments.
- Par exemple, utiliser le sélecteur **p a** en **CSS** va nous permettre d'appliquer des styles à tous les éléments **a** contenus dans des éléments **p** et seulement aux éléments **a** contenus dans des éléments **p**

Code **HTML**:

```
<> index.html X # style.css
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title>ODC-GUINEE</title>
6          <link rel="stylesheet" href="style.css">
7      </head>
8      <body>
9          <h1>Un titre de niveau 1</h1>
10         <p>Un premier paragraphe</p>
11         <p>Un autre paragraphe avec un <a href="#">lien</a></p>
12         <ul>
13             <li><a href="#">Hommes</a></li>
14             <li>Femmes</li>
15         </ul>
16     </body>
17 </html>
```

Code **CSS**:

```
# style.css > ...
1  p a{
2      background-color: orangered;
3  }
4
```

Résultat

Un titre de niveau 1

Un premier paragraphe

Un autre paragraphe avec un [lien](#)

- [Hommes](#)
- Femmes

4. Appliquer des styles aux enfants directs d'un autre élément

- Nous allons également pouvoir cibler uniquement un élément ou un groupe d'éléments enfants directs d'un autre élément en utilisant le signe de supériorité stricte ou le caractère chevron fermant `>`.
- Un élément est un enfant direct ou « descendant direct » d'un autre élément s'il est directement contenu dans celui-ci.
- Par exemple, nous allons pouvoir appliquer des styles à tous les liens (éléments `a`) qui sont des enfants directs de l'élément `body` et uniquement à ceux-ci :



Code HTML:

```
< index.html x # style.css
< index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title>ODC-GUINEE</title>
6          <link rel="stylesheet" href="style.css">
7      </head>
8      <body>
9          <h1>Un titre de niveau 1</h1>
10         <a href="#">Un lien</a>
11         <p>Un premier paragraphe</p>
12         <p>Un autre paragraphe avec un <a href="#">lien</a></p>
13         <ul>
14             <li><a href="#">Hommes</a></li>
15             <li>Femmes</li>
16         </ul>
17     </body>
18 </html>
```

Code CSS:

```
# style.css > ...
1  body > a{
2      background-color: orangered;
3  }
4
```

Résultat

Un titre de niveau 1

Un lien

Un premier paragraphe

Un autre paragraphe avec un lien

- Hommes
- Femmes

5. Sélectionner l'élément suivant directement un autre élément en CSS

- Le **CSS** va nous permettre de cibler encore plus précisément un élément en ciblant l'élément suivant directement un autre élément grâce au caractère **+**.
- Par exemple, on va pouvoir cibler les éléments **a** (liens) suivant directement un élément **p** :



Exemples pratiques :

Code **HTML**:

```
<> index.html ● # style.css
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <title>ODC-GUINEE</title>
6    <link rel="stylesheet" href="style.css">
7  </head>
8  <body>
9    <h1>Un titre de niveau 1</h1>
10   <a href="#">Un lien</a>
11   <p>Un premier paragraphe</p>
12   <a href="#">Un autre lien</a>
13   <p>Un autre paragraphe avec un <a href="#">lien</a></p>
14   <ul>
15     <li><a href="#">Hommes</a></li>
16     <li>Femmes</li>
17   </ul>
18
19 </body>
20 </html>
```

Code **CSS**:

```
<> index.html # style.css X
# style.css > ...
1  /* Applique une couleur de fond orange-rouge à tout élément a
2  suivant directement un élément p */
3  p + a{
4    background-color: ■ orangered;
5  }
6
```

Résultat

Un titre de niveau 1

[Un lien](#)

Un premier paragraphe

[Un autre lien](#)

Un autre paragraphe avec un [lien](#)

- [Hommes](#)
- Femmes

6. Sélectionner tous les éléments suivant un autre élément en CSS

- Le caractère tilde ~ va nous être plus permissif que le caractère + en nous permettant cette fois-ci de sélectionner tous les éléments déclarés après un autre élément en HTML de même niveau (c'est-à-dire possédant le même parent direct).
- Par exemple, on va pouvoir cibler tous les éléments a placés après un élément p et qui sont de même niveau :



Code HTML:

```
<> index.html X # style.css
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title>ODC-GUINEE</title>
6          <link rel="stylesheet" href="style.css">
7      </head>
8      <body>
9          <h1>Un titre de niveau 1</h1>
10         <a href="#">Un lien</a>
11         <p>Un premier paragraphe</p>
12         <a href="#">Un autre lien</a>
13         <p>Un autre paragraphe avec un <a href="#">lien</a></p>
14         <ul>
15             <li><a href="#">Hommes</a></li>
16             <li>Femmes</li>
17         </ul>
18         <a href="#">Un dernier lien</a>
19     </body>
20 </html>
21
```

Code CSS:

```
<> index.html # style.css X
# style.css > ...
1  /* Applique une couleur de fond orange-rouge à tout élément a
2  suivant directement un élément p */
3  p ~ a{
4      background-color: ■ orangered;
5  }
```

Résultat

Un titre de niveau 1

[Un lien](#)

Un premier paragraphe

[Un autre lien](#)

Un autre paragraphe avec un [lien](#)

- [Hommes](#)
- Femmes

[Un dernier lien](#)

Résumé des caractères spéciaux permettant de combiner des sélecteurs et signification

Vous pourrez trouver ci-dessous un résumé des différents caractères vus dans cette partie et qui vont nous permettre de combiner des sélecteurs CSS simples afin d'en créer des plus complexes :

Sélecteur CSS	Signification
*	Sélectionne tous les éléments
E, F	Sélectionne tous les éléments de type E et de type F
E F	Sélectionne tous les éléments F à l'intérieur des éléments E
E > F	Sélectionne les éléments F enfants directs des éléments E
E + F	Sélectionne tout élément F placé directement après un élément E
E ~ F	Sélectionne tout élément F placé après un élément E dans la page

Ordre d'application (cascade) et des règles en CSS

Comprendre l'importance d'établir un ordre d'application des règles **CSS** : le problème des conflits.

- Pour comprendre les mécanismes fondamentaux de cascade et d'héritage en **CSS**, il faut avant tout comprendre ce qu'est un conflit **CSS**.
- Parfois, plusieurs sélecteurs différents vont nous permettre d'appliquer des styles **CSS** à un même élément.
- Imaginons par exemple un élément **p** auquel on attribuerait un attribut **class** et un attribut **id**. Nous allons pouvoir appliquer des styles **CSS** à cet élément de trois façons évidentes différentes :
 - en utilisant un sélecteur élément ;
 - en le ciblant via son attribut **class** ;
 - en le ciblant via son attribut **id**.

Code HTML:

```
<> index.html X # style.css
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title>ODC-GUINEE</title>
6          <link rel="stylesheet" href="style.css">
7      </head>
8      <body>
9          <!-- Comprendre : le problème des conflits. -->
10         <h1 class="bigorange">Un titre de niveau 1</h1>
11         <p>Un premier paragraphe</p>
12         <p class="bigorange">Un autre paragraphe</p>
13         <p class="bigorange" id="green">Un troisième paragraphe</p>
14         <p style="color: purple;" class="bigorange" id="grey">Un quatrième paragraphe</p>
15     </body>
16 </html>
```

Code CSS:

```
<> index.html # style.css X
# style.css > ...
1  p{
2      color: red;
3      text-decoration: underline;
4  }
5  .bigorange{
6      color: orange;
7      font-size: 25px;
8  }
9  #green{
10     color: green;
11 }
12 #grey{
13     color: grey;
14 }
```

Résultat

Un titre de niveau 1

Un premier paragraphe

Un autre paragraphe

Un troisième paragraphe

Un quatrième paragraphe

Le mécanisme de cascade CSS

- L'ordre de préférence et d'application d'un style va dépendre de trois grands facteurs qui vont être :
 1. La présence ou non du mot clef **!important** ;
 2. La précision du sélecteur ;
 3. L'ordre de déclaration dans le code ;
- A noter que ces trois facteurs vont être analysés dans l'ordre donné et que le premier va primer sur le deuxième qui va primer sur le dernier : par exemple, si une règle utilise la syntaxe **!important** elle sera jugée comme prioritaire peu importe la précision du sélecteur ou sa place dans le code.

Le mot clef !important

La mot clef **!important** sert à forcer l'application d'une règle **CSS**. La règle en question sera alors considérée comme prioritaire sur toutes les autres déclarations et le style sera appliqué à l'élément concerné.

Code HTML:

```
<> index.html X # style.css
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title>ODC-GUINEE</title>
6          <link rel="stylesheet" href="style.css">
7      </head>
8      <body>
9          <!-- Comprendre : le problème des conflits. -->
10         <h1 class="bigorange">Un titre de niveau 1</h1>
11         <p>Un premier paragraphe</p>
12         <p class="bigorange">Un autre paragraphe</p>
13         <p class="bigorange" id="green">Un troisième paragraphe</p>
14         <p style="color: purple;" class="bigorange" id="grey">Un quatrième paragraphe</p>
15     </body>
16 </html>
```

Code CSS:

```
<> index.html # style.css X
# style.css > ...
1  p{
2      color: red !important;
3      text-decoration: underline;
4  }
5  .bigorange{
6      color: orange;
7      font-size: 25px;
8  }
9  #green{
10     color: green;
11 }
12 #grey{
13     color: grey;
14 }
```

Résultat

Un titre de niveau 1

Un premier paragraphe

Un autre paragraphe

Un troisième paragraphe

Un quatrième paragraphe

3. L'ordre de déclaration dans le code.

- Le troisième et dernier critère qui va nous permettre de définir quel style doit primer sur tel autre et doit donc être appliqué à un élément va tout simplement être l'ordre d'écriture d'une règle dans le code.
- Ce critère va être utilisé dans le cas où plusieurs sélecteurs concurrents définissent le comportement d'une même propriété et ont la même importance et la même spécificité.
- La règle ici est très simple : c'est la dernière déclaration dans le code qui primera sur des déclarations précédentes.

Regardez plutôt l'exemple suivant :



Code HTML:

```
<> index.html X # style.css
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title>ODC-GUINEE</title>
6          <link rel="stylesheet" href="style.css">
7          <style>
8              h1{
9                  color: orange;
10                 font-size: 25px;
11             }
12         </style>
13     </head>
14     <body>
15         <!-- L'ordre d'écriture des règles -->
16         <h1 class="petit">Un titre de niveau 1</h1>
17         <p class="petit grand">Un premier paragraphe</p>
18         <p class="orange bleu">Un autre paragraphe</p>
19     </body>
20 </html>
```

Code CSS:

```
<> index.html # style.css
# style.css > ...
1  h1{
2      color: blue;
3  }
4  .petit{
5      font-size: 14px;
6  }
7  .grand{
8      font-size: 28px;
9  }
10 .bleu{
11     color: blue;
12 }
13 .orange{
14     color: orange;
15 }
```

Résultat

Un titre de niveau 1

Un premier paragraphe

Un autre paragraphe

• Largeur (width) et hauteur (height) de la boîte de contenu des éléments HTML

- Les éléments **HTML** peuvent être affichées de deux grandes façons différentes : soit sous forme de **bloc**, soit en **ligne**.
- Les dimensions par défaut du contenu des éléments **HTML** vont avant tout être déterminées par le type d’affichage des éléments : en effet, les éléments de type **block** occuperont par défaut toute la largeur disponible dans leur élément parent tandis que les éléments de type **inline** n’occuperont que la largeur nécessaire à leur contenu.
- Vous pouvez ainsi déjà retenir ici que nous n’allons pas pouvoir modifier la taille de l’espace pris par le contenu des éléments de type **inline** avec les propriétés **width** et **height** : les valeurs qu’on va pouvoir définir vont tout simplement être ignorées.
- En effet, le principe de base d’un élément de type **inline** est que l’espace pris par sa boîte « contenu » soit toujours égal à l’espace strictement nécessaire à l’affichage de ce contenu.
- Regardez l’exemple ci-dessous pour vous en convaincre. Je vous rappelle ici qu’un élément **p** possède un type d’affichage **block** tandis qu’un élément **span** possède un type d’affichage **inline**.

Code **HTML**:

```
index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title>ODC-GUINEE</title>
6          <link rel="stylesheet" href="style.css">
7      </head>
8      <body>
9          <h1>Un titre de niveau 1</h1>
10         <p class="p1">Un premier paragraphe</p>
11         <p class="p2">Un autre paragraphe</p>
12         <span class="sp1">Un élément span</span>
13         <span class="sp2">Un autre span</span>
14     </body>
15 </html>
```

Code **CSS**:

```
index.html  style.css  X
style.css > ...
1  .p1{
2      background-color: lightBlue;
3      width: 400px;
4      height: 100px;
5  }
6  .p2{
7      background-color: lightGreen;
8  }
9  .sp1{
10     background-color: orange;
11     width: 400px;
12     height: 100px;
13 }
14 .sp2{
15     background-color: yellow;
16 }
```

Résultat

Un titre de niveau 1

Un premier paragraphe

Un autre paragraphe

Un élément span Un autre span

• Gestion des marges internes ou padding en CSS

- On va en effet pouvoir passer entre 1 et 4 valeurs à la propriété raccourcie **padding** :
- En passant une valeur à **padding**, la valeur passée définira le comportement des 4 marges intérieures de l'élément ;
- En passant deux valeurs à **padding**, la première valeur passée définira le comportement des marges intérieures supérieure et inférieure de l'élément tandis que la seconde valeur définira le comportement des marges intérieures gauche et droite de l'élément ;
- En passant trois valeurs à **padding**, la première valeur passée définira le comportement de la marge interne supérieure, la deuxième définira le comportement des marges intérieures gauche et droite tandis que la troisième définira le comportement de la marge interne basse ;
- En passant quatre valeurs à **padding**, la première valeur passée définira le comportement de la marge interne supérieure, la deuxième définira le comportement de la marge interne droite, la troisième celui de la marge interne basse et la quatrième celui de la marge interne gauche.

- Si on choisit d'utiliser les propriétés `padding-top`, `padding-left`, `padding-bottom` et `padding-right`, il suffit de suivre le nom pour comprendre à quelle marge interne est liée chaque propriété (top = haute, left = gauche, bottom = basse, right = droite).

• Gestion des bordures avec la propriété CSS `border`

- Les bordures vont être définies par trois caractéristiques en CSS : une épaisseur (ou largeur), un style et une couleur.
- Nous allons pouvoir définir ces différentes caractéristiques d'un coup au sein de la propriété raccourcie `border` ou les définir une à une avec chacune des propriétés de type `border-`. Plus précisément :
- La propriété `border-width` va nous permettre de définir la largeur (ou « l'épaisseur ») d'une bordure ;
- La propriété `border-style` va nous permettre de définir le style d'une bordure ;
- La propriété `border-color` va nous permettre de définir la couleur d'une bordure.

- Définir la largeur ou l'épaisseur d'une bordure

- Nous allons pouvoir utiliser différents types de valeurs pour définir la largeur d'une bordure
 - ❖ Une valeur de type « mot clef » à choisir parmi **thin** (bordure fine), **medium** (bordure moyenne) et **thick** (bordure épaisse) ;
 - ❖ Une valeur de type « longueur » en **px** ou en **em** par exemple.
 - ❖ Généralement, nous utiliserons des unités en px pour définir la largeur de nos bordures.

❑ Choisir le style d'une bordure

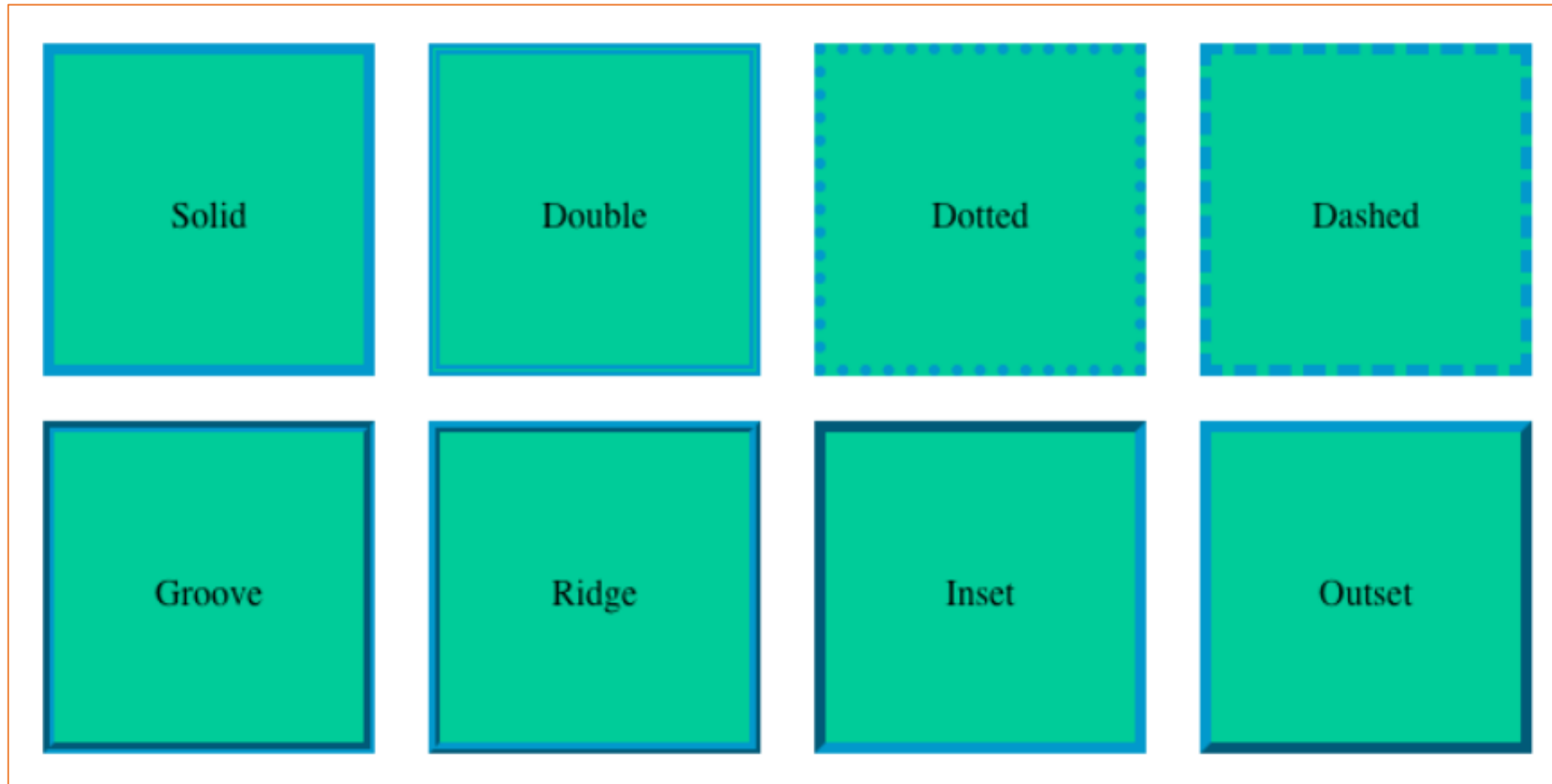
- Le « style » d'une bordure correspond à son aspect : une bordure peut prendre la forme d'un simple trait, d'un trait double, ou être constituée de pointillés, avoir un effet 3D, etc.
- Pour définir le style d'une bordure, nous allons devoir choisir parmi les mots clefs suivants :



- Chaque mot clef va créer un type de bordure différent.

Valeur	Description
solid	Bordure solide simple (un trait)
double	Bordure solide double
dotted	Bordure en pointillés
dashed	Bordure constituée de tirets
groove	Bordure incrustée avec effet 3D. L'effet produit est l'inverse de ridge
ridge	Bordure en relief avec effet 3D. L'effet produit est l'inverse de groove
inset	La bordure donne l'effet que la boîte représentant l'élément est enfoncée dans la page. L'effet produit est l'inverse de outset
outset	La bordure donne l'effet que la boîte représentant l'élément est en relief par rapport au reste de la page. L'effet produit est l'inverse de inset

- Voici à quoi correspond chaque effet visuellement :



JOUR 4: STRUCTURE D'UN SITE WEB

• Gestion des marges externes ou margin en CSS

- Grâce à la propriété CSS **margin** nous allons pouvoir ajouter des marges externes à un élément. Cette propriété est en fait la notation raccourcie des propriétés **margin-top**, **margin-left**, **margin-bottom** et **margin-right** qui vont servir à définir les marges externes de chaque côté d'un élément.
- Ces propriétés vont pouvoir accepter différents types de valeurs :
- Des valeurs de type longueur, généralement en **px** ou en **em** ;
- Des valeurs de type pourcentage. Dans ce cas, le **%** indiqué est calculé par rapport à la taille de l'élément parent ;
- Le mot clef **auto**. Ce mot clef va surtout nous servir pour centrer des blocs.
- Notez qu'à la différence de la propriété **padding**, nous allons pouvoir passer des valeurs négatives à **margin** pour créer des marges externes négatives même si en pratique cela est généralement déconseillé pour les problèmes d'ergonomie et de consistance du design que ça peut causer au niveau de la page.

➤ **NB** : On ne va pas pouvoir appliquer de marges haute ou basse aux éléments de type **inline** comme c'était déjà le cas pour les marges internes. En revanche, les marges externes droite et gauche s'appliquent normalement.

On va en effet pouvoir passer entre 1 et 4 valeurs à la propriété raccourcie **margin** :

- En passant une valeur à **margin**, la valeur passée définira le comportement des 4 marges extérieures de l'élément ;
- En passant deux valeurs à **margin**, la première valeur passée définira le comportement des marges extérieures supérieure et inférieure de l'élément tandis que la seconde valeur définira le comportement des marges extérieures gauche et droite de l'élément ;
- En passant trois valeurs à **margin**, la première valeur passée définira le comportement de la marge externe supérieure, la deuxième définira le comportement des marges extérieures gauche et droite tandis que la troisième définira le comportement de la marge externe basse ;
- En passant quatre valeurs à **margin**, la première valeur passée définira le comportement de la marge externe supérieure, la deuxième définira le comportement de la marge externe droite, la troisième celui de la marge externe basse et la quatrième celui de la marge externe gauche.

• Créer des bordures arrondies avec border-radius en CSS

- La propriété **border-radius** va prendre deux valeurs représentant la dimension du « rayon » sur l'axe des X (axe horizontal) et la dimension du « rayon » sur l'axe des Y (axe vertical) d'une ellipse qui va servir à définir la forme de l'arrondi.
- Ces deux valeurs doivent être séparées par un slash comme ceci : **border-radius: X/Y**. La deuxième valeur est facultative et si elle est omise elle sera considérée comme égale à la première par défaut.
- Ces deux valeurs vont pouvoir être des valeurs de type longueur (en **px** par exemple) ou des pourcentages (%).
- Prenons directement un premier exemple pour illustrer tout ça :



Code HTML:

```
index.html X style.css
index.html > ...
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8">
5     <title>ODC-GUINEE</title>
6     <link rel="stylesheet" href="style.css">
7   </head>
8   <body>
9     <div><p>Un paragraphe sans bordure</p></div>
10    <div><p class="bordure">border: 2px solid black</p></div>
11    <div><p class="arrondi1">border-radius : 100px / 20px</p></div>
12    <div><p class="arrondi2">border-radius: 50px</p></div>
13    <div><p class="arrondi2 bordure">border-radius: 50px avec bordure</p></div>
14    <div><p class="arrondi3">border-radius: 25%</p></div>
15  </body>
16 </html>
```

Code CSS:

```
index.html style.css X
style.css > ...
1 div{
2   width: 80%;
3   margin: 20px auto; /*Centre les div dans la page*/
4   background-color: aqua; /*Couleur de fond*/
5 }
6 p{
7   width: 100%;
8   height: 100%;
9   box-sizing: border-box;
10  text-align:center; /*Texte centré horizontalement*/
11  line-height: 100px; /*Texte centré verticalement*/
12  background-color: goldenrod; /*Couleur de fond*/
13 }
14 .bordure{
15   border: 2px solid black;
16 }
17 .arrondi1{
18   border-radius : 100px / 20px;
19 }
20 .arrondi2{
21   border-radius: 50px; /*Identique à border-radius: 50px/50px*/
22 }
23 .arrondi3{
24   border-radius: 25%; /*Identique à border-radius: 25%/25%*/
25 }
```

Résultat

Un paragraphe sans bordure

border: 2px solid black

border-radius : 100px / 20px

border-radius: 50px

border-radius: 50px avec bordure

border-radius: 25%

- Créer des tableaux avec l'élément HTML table

- Les tableaux en **HTML** vont nous permettre de présenter des données de manière organisée et sous une certaine forme pour les structurer et les rendre compréhensibles pour les navigateurs, moteurs de recherche et utilisateurs.
- Un tableau est un ensemble de lignes et de colonnes. L'intersection entre une ligne et une colonne est une cellule de tableau.
- Pour créer un tableau fonctionnel en **HTML**, nous allons devoir utiliser à minima 3 éléments :
- Un élément **table** (« tableau » en français) qui va définir le tableau en soi ;
- Des éléments **tr**, pour « table row » ou « ligne de tableau » en français qui vont nous permettre d'ajouter des lignes dans notre tableau ;
- Des éléments **td**, pour « table data » ou « donnée de tableau » en français qui vont nous permettre d'ajouter des cellules dans nos lignes et ainsi de créer automatiquement de nouvelles colonnes.

- L'élément **HTML table** va représenter le tableau en soi. Cet élément est composé d'une paire de balises ouvrante `<table>` et fermante `</table>` au sein desquelles nous allons placer les différents autres éléments de notre tableau. Les éléments **tr** et **td** sont également représentés sous la forme d'une paire de balises avec leur contenu entre les balises.

Prenom	Nom	Age	Sexe	Adresse	Telephone	Mail
Oumar	Bah	23	Masculin	Conakry	123456789	ouamr@gmail.com
Saliou	Diallo	25	Masculin	Conakry	987654321	saliou@gmail.com
Aliou	Barry	30	Masculin	Conakry	456789123	aliou@gmail.com
Maimouna	Bangoura	28	Feminin	Conakry	321654987	maimouna@gmail.com

1 - Structuration d'un site web:

- Utilisation de balises **HTML** pour organiser le contenu (**header**, **nav**, **section**, **article**, **footer**).

Le HTML et la sémantique

- Grâce au **HTML**, nous allons pouvoir marquer nos différents contenus avec de nombreux éléments qui vont permettre de leur donner du sens. Utiliser les « bons » éléments à chaque fois et ceux qui font le plus de sens va ainsi être essentiel pour que notre page soit la plus lisible et la plus compréhensible possible pour les différents robots et programmes.
- Le **HTML5** nous offre justement une série de nouveaux éléments dit « structurants » et qui vont nous permettre de préciser le sens de certains contenus.
- Certains de ces éléments vont notamment pouvoir être utilisés à la place des classiques **div** et **span** qui sont deux conteneurs génériques c'est-à-dire des éléments qui n'apportent aucun sens au contenu.

Liste et description des éléments structurants

Le **HTML5** apporte les éléments structurants suivants :

Nom de l'élément	Description
header	Représente l'en-tête ou le haut d'une page
nav	Représente un menu de navigation dans une page
main	Représente le contenu principal de l'élément body
section	Représente une section dans une page, c'est-à-dire un groupement thématique de contenu
article	Représente une partie de page qui se suffit à elle-même, comme un post sur un blog par exemple
aside	Représente une partie de contenu non lié au reste
footer	Représente le pied d'une page

L'élément header

- L'élément **header** représente l'en-tête de nos pages. Cet élément va généralement contenir le logo du site et le menu principal de navigation. Dans le cas d'un blog, si nous avons un article de blog qui est placé dans un élément **article**, on pourra également utiliser un élément **header** dans l'élément **article** et y placer les informations relatives à l'auteur, au temps de lecture, etc.

L'élément nav

- L'élément **nav** représente un menu de navigation. Nous allons par exemple pouvoir utiliser cet élément pour entourer le menu de nos pages. Cela va permettre d'indiquer aux navigateurs et moteurs de recherche que notre élément **ul** (utilisé pour créer un menu) n'est pas qu'une liste mais bien un menu de navigation.

L'élément main

- L'élément **main** représente le contenu principal ou « majoritaire » du body. La zone de contenu principale d'un document comprend un contenu unique à ce document et exclut le contenu répété sur un ensemble de documents, tels que des liens de navigation de site, des informations de copyright, des logos de site et des bannières.

L'élément article

- L'élément **article** représente une partie indépendante d'une page. Nous utiliserons cet élément pour entourer du contenu qui pourrait être extrait de notre page et distribué ailleurs tout en continuant à faire du sens.

L'élément section

- L'élément **section** représente à nouveau une partie d'une page cohérente mais qui ne va pas forcément faire du sens si on l'extrait seule de la page pour l'intégrer ailleurs. L'élément **section** est à mi-chemin entre les éléments **article** et **div** et nous pourrions nous en servir pour organiser nos pages.

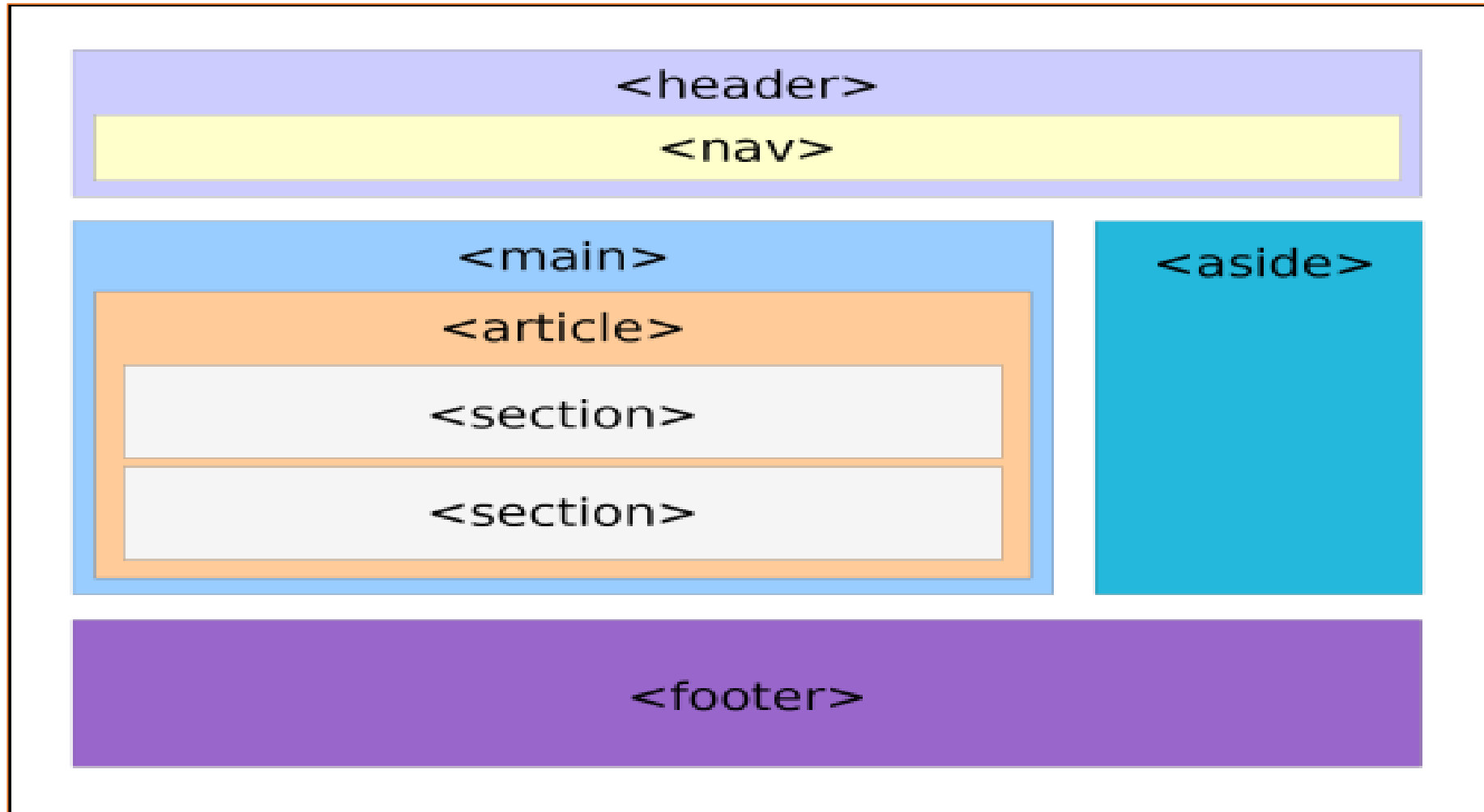
L'élément aside

- L'élément **aside** permet d'indiquer un contenu en marge du reste de la page, c'est-à-dire différent du reste de la page. Imaginons par exemple que nous ayons un blog et que chaque article de blog se trouve dans un élément **article**.

L'élément footer

- L'élément **footer** va avoir un rôle et une signification sémantique identiques à l'élément **header**. Comme son nom l'indique, nous utiliserons cet élément pour entourer les informations de pied de page (notice de copyright, menu de navigation bas, etc.).

Voici par exemple une utilisation des éléments structurants **HTML.**



Travaux pratiques

- Creation de menus de navigation en intégrant des icônes **Font Awesome**.
- Ajoutez la bibliothèque Font Awesome

Première étape : placer ce lien dans l'élément **head**

```
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.15.3/css/all.min.css">
```

Deuxième étape : installer l'extension font awesome gallery via vs code



- Responsive design:

Lorsqu'on parle de « responsive design » ou de « design adaptable » en français, on fait référence à l'idée selon laquelle un site web devrait s'afficher aussi bien sur un écran de PC que sur un écran de smartphone ou sur n'importe quel type d'appareil.

Aujourd'hui, nous pouvons utiliser principalement trois méthodes pour répondre aux défis amenés par les différentes tailles d'écran. On peut :

- Créer une application dédiée pour les mobiles ;
- Créer une « copie mobile » de notre site en utilisant l'initiative AMP (« Accelerated Mobile Pages ») de Google ;
- Utiliser les Media Queries ou requêtes media.

L'idée de base du responsive design est qu'un site web devrait présenter les mêmes informations quel que soit l'appareil qui l'affiche, en les réarrangeant pour conserver la meilleure ergonomie possible.

• Définition des Media Queries

- Les Media Queries vont nous permettre d'appliquer certaines règles **CSS** de manière conditionnelle. Par exemple, on va pouvoir définir une largeur pour un élément pour certaines tailles d'écrans et une largeur différente pour le même élément pour d'autres tailles d'écran.
- Cela va nous permettre d'afficher des pages avec des organisations différentes selon la taille de l'écran d'un visiteur.
- Avec les Media Queries, nous allons pouvoir appliquer des règles **CSS** totalement différentes selon les tailles d'écrans

NB : Avant d'étudier les Media Queries et la création de design responsives en profondeur, il est important de comprendre le rôle de la balise `<meta name= "viewport">`.

• L'élément HTML meta

- L'élément HTML **meta** est utilisé pour définir des métadonnées pour un document HTML.
- Une métadonnée est une donnée qui ne va pas être affichée sur la page mais qui va pouvoir servir aux différents robots pour comprendre et afficher la page.
- On va pouvoir ajouter différents attributs à l'élément **meta** qui vont nous servir à spécifier différents types de données.
- Les attributs les plus courants de l'élément **meta** vont être les attributs **charset**, **name** et **content**.
- Nous connaissons déjà l'attribut **charset** qui sert à indiquer le jeu de caractères ou l'encodage de notre document.

L'attribut **name** va nous permettre d'indiquer le type de métadonnées que l'on souhaite passer. Cet attribut va aller de pair avec l'attribut content qui va lui nous permettre de passer une métadonnée en soi.

L'attribut **name** va notamment pouvoir prendre les valeurs suivantes :

author : la valeur passée à **content** sera considérée comme étant le nom de l'auteur du document ;

description : la valeur passée à **content** pourra être utilisée par les moteurs de recherche comme extrait pour décrire le sujet de notre page ;

viewport : la valeur passée à **content** va nous permettre d'indiquer comment le navigateur doit afficher la page sur différents appareils.

Projet à réaliser: